



Lab 1

Dataset: stats_case_study_dataset_v5.csv (download provided)

Topics (only these):

- Descriptive vs Inferential; Types of data; Scales (Nominal/Ordinal/Interval/Ratio)
- Population vs Sample; Sampling (Random/Stratified/Cluster/Systematic)
- Data sources (Survey/Experiment/Observational); Biases in collection
- Central tendency: Mean (Arithmetic/Weighted), Median, Mode; When to use
- Impact of outliers on central tendency (salary, sales)
- Dispersion: Range, Variance & Std (sample vs population), MAD, CV
- Comparing datasets via dispersion

Instructions: Each exercise appears in a markdown cell. Implement your answer in the empty code cell **directly below** it.

Setup

Import libraries and load the dataset here.

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

df = pd.read_csv(r"C:\Users\uktiw\Downloads\stats_case_study_dataset_v5.csv")
df.head()
```

```
Out[1]:
```

	id	department	region	gender	education_level	years_experience	age	is_n
0	1	Tech	West	Male	Bachelors	5.4	39	
1	2	Operations	South	Female	Bachelors	2.1	40	
2	3	Finance	North	Male	Bachelors	16.8	41	
3	4	Tech	South	Male	High School	6.7	36	
4	5	Sales	North	Female	High School	6.6	31	

Foundations: Concepts & Data Types

Exercise 1: In 1–2 sentences, distinguish **Descriptive** vs **Inferential** statistics using this dataset.

Descriptive statistics - It can measure of the central tendency of the dataset with Mean , Median, Mode & spread of Data

Inferential statistics - It is used to make inferences about a population based on a sample of data drawn from that population.

Exercise 2: Classify variables as **Qualitative** (categorical) or **Quantitative** (numeric): department, region, gender, education_level, years_experience, age, is_manager, performance_rating, salary, monthly_sales, work_hours_per_week.

Qualitative= department, region, gender, education_level, is_manager

Quantitative= age , is_manager, work_hours_per_week ,performance_rating,salary,monthly_sales

Is_manager comes in Qualitative because there are only two values in is_manager 0 and 1 which means Yes and No, so we can consider it in Qualitative not Quantitative.

Exercise 3: Assign **scales of measurement** (Nominal / Ordinal / Interval / Ratio) to each variable above.

Nominal - department,region,gender,is_manager

Ordinal - education_level, performance_rating

Interval - N/A

Ratio - salary, monthly_sales, work_hours_per_week, years_experience, age

Exercise 4: Explain **Population vs Sample** in this lab context.

Population = Total no of student in the class = 100

Sample = Total no of student in the class who are selected for the survey = 20

Exercise 5: Give an example for each **data source: Survey, Experiment, Observational study** relevant to this dataset.

Survey = Done a survey by collecting data from employees about Job satisfaction and Work-life balance.

Experiment = Done an experiment by randomly assigning employees to two groups: one group received a new flexible work schedule, while the other group continued with their regular schedule. After a month, we measured their job satisfaction and work-life balance.

Observational Study= In the observational study , we observed that employee with new flexible work schedule had higher job satisfaction and better work-life balance compared to those with regular schedule.

Exercise 6: List **two biases** that could affect data collection here.

Sampling Bias - If employees are selected from only one Department.

Response Bias - If employees are not comfortable sharing their true feelings about the company.

Sampling Methods: Random, Stratified, Cluster, Systematic

Exercise 1: Load the dataset from `/mnt/data/stats_case_study_dataset_v5.csv` into `df` and show `df.head(3)`.

```
In [2]: df.head(3)
```

```
Out[2]:
```

	id	department	region	gender	education_level	years_experience	age	is_n
0	1	Tech	West	Male	Bachelors	5.4	39	
1	2	Operations	South	Female	Bachelors	2.1	40	
2	3	Finance	North	Male	Bachelors	16.8	41	

Exercise 2: Draw a **Simple Random Sample** of 40 rows (without replacement) and compute the sample mean salary.

```
In [3]: sample_row = df.sample(n=40, random_state= 42, replace = False)
sample_mean = sample_row['salary'].mean()
print(f"Sample Mean: {sample_mean}")
```

Sample Mean: 68035.225

Exercise 3: Perform **Stratified Sampling** by `department`: sample 10% from each stratum (at least 2 per department). Compute the combined sample mean

salary.

```
In [4]: def stratified_sampling(group):
        sample_size = int(len(group)*0.1) # Adjust the fraction as needed
        return group.sample(sample_size , random_state = 2)
        stratified_sample = df.groupby('department').apply(stratified_sampling)
        stratified_sample['salary'].mean()
```

C:\Users\uktiw\AppData\Local\Temp\ipykernel_19772\327061004.py:4: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
stratified_sample = df.groupby('department').apply(stratified_sampling)
```

Out[4]: np.float64(64455.59090909091)

Exercise 4: Create a **Cluster Sample** by **region**: randomly pick **1 region** and include all rows from that region. Compute its mean monthly_sales.

```
In [5]: selected_region = df['region'].sample(1, random_state=4).iloc[0]
        selected_df= df[df['region'] == selected_region]
        selected_df['monthly_sales'].mean()
```

Out[5]: np.float64(13824.373134328358)

Exercise 5: Perform **Systematic Sampling**: sort by **id**, choose start=3, take every 5th row. Report the sample size.

```
In [6]: df.iloc[3::5].head()
```

Out[6]:

	id	department	region	gender	education_level	years_experience	age	is_
3	4	Tech	South	Male	High School	6.7	36	
8	9	HR	North	Male	Bachelors	4.2	34	
13	14	Sales	South	Female	High School	6.9	32	
18	19	Tech	North	Female	Masters	11.5	30	
23	24	Tech	West	Female	Bachelors	3.4	34	

Central Tendency: Mean (Arithmetic & Weighted), Median, Mode, When to Use

Exercise 1: Compute **mean**, **median**, and **mode** of **salary** for the full dataset.

```
In [7]: print(df['salary'].mean())
```

```
print(df['salary'].median())
print(df['salary'].mode())
```

```
65316.229166666664
60493.5
0      73527.0
Name: salary, dtype: float64
```

Exercise 2: Compute a **weighted mean** of `monthly_sales` using `work_hours_per_week` as weights.

```
In [8]: df.head(3)
```

```
Out[8]:
```

	id	department	region	gender	education_level	years_experience	age	is_n
0	1	Tech	West	Male	Bachelors	5.4	39	
1	2	Operations	South	Female	Bachelors	2.1	40	
2	3	Finance	North	Male	Bachelors	16.8	41	

```
In [9]: weighted_mean_monthly_sales = np.average(df['monthly_sales'],
weights=df['work_hours_per_week'])
print(weighted_mean_monthly_sales )
```

```
11911.638701165286
```

```
In [10]: normal_mean = df['monthly_sales'].mean()
print(normal_mean)
```

```
12063.6625
```

Exercise 3: Compute department-wise central tendency of `monthly_sales` (mean & median).

```
In [11]: df.groupby('department')['monthly_sales'].agg(['mean', 'median'])
```

```
Out[11]:
```

	mean	median
department		
Finance	6469.558824	6276.5
HR	6243.111111	5748.0
Operations	5731.970588	6096.5
Sales	23053.925926	22235.0
Tech	6945.234375	6163.5

Exercise 4: In 1-2 comment lines, note **when to use** mean vs median vs mode for salary.

When to use

Mean: When data is normal data without outliers, and show normal distribution, then we can use mean.

Median: When data is skewed or has outliers, then we can use median to get a better representation of the central tendency.

Mode: When we want to find the most common salary in dataset, especially for categorical data, then we can use mode.

Exercise 5: For the **Sales** department only, compute mean & median of `monthly_sales` and compare briefly in a comment.

```
In [12]: sales_department= df[df['department']=='Sales']
mean =sales_department['monthly_sales'].mean()
median= sales_department['monthly_sales'].median()
print(f"Sales Department - Mean: {mean}, Median: {median}")
```

Sales Department - Mean: 23053.925925925927, Median: 22235.0

Insight: Mean is greater than median so the graph of the monthly sales would be right skewed

Impact of Outliers on Central Tendency

Exercise 1: Compute `salary` mean and median with all rows.

```
In [13]: df['salary'].agg(['mean', 'median']).reset_index()
```

```
Out[13]:
```

	index	salary
0	mean	65316.229167
1	median	60493.500000

Exercise 2: Remove the **top 1%** salaries and recompute mean & median. Comment on the change.

```
In [14]: sort_df = df.sort_values(by='salary', ascending=False)
threshold= sort_df['salary'].quantile(0.99)
filtered_df= sort_df[sort_df['salary'] <= threshold]
filtered_df['salary'].agg(['mean', 'median']).reset_index()
```

```
Out[14]:
```

	index	salary
0	mean	63851.042194
1	median	60452.000000

```
In [15]: normal_mean_value = df['salary'].mean()
normal_median_value = df['salary'].median()
print(f"Normal Mean: {normal_mean_value}")
print(f"Normal Median: {normal_median_value}")
```

Normal Mean: 65316.229166666664

Normal Median: 60493.5

Insights: After Removing Top 1% of the salaries and computing there mean and median , both mean and median reduces from original, because we have removed the top outliers

Exercise 3: Compute a **10% trimmed mean** for salary (drop lowest 5% and highest 5%).

```
In [16]: lower = df['salary'].quantile(0.05)
upper = df['salary'].quantile(0.95)
trimmed_df = df[(df['salary']>=lower) & (df['salary']<=upper)]
trimmed_mean = trimmed_df['salary'].mean()
print(f'trimmed_mean: {trimmed_mean}')
```

trimmed_mean: 63736.34722222222

Dispersion: Range, Variance/Std (Population vs Sample), MAD, CV

Exercise 1: Compute the **range** of salary .

```
In [17]: #range = max- min
salary_range = df['salary'].max() - df['salary'].min()
print(f"Salary Range: {salary_range}")
```

Salary Range: 184625.0

Exercise 2: Compute **population variance** and **population std dev** of salary .

```
In [18]: population_variance = df['salary'].var(ddof=0)
print(f"Population Variance: {population_variance}")
population_std = df['salary'].std(ddof=0)
print(f"Population_std: {population_std}")
```

Population Variance: 472165047.90998167

Population_std: 21729.35912331474

Exercise 3: Compute **sample variance** and **sample std dev** of `salary` .

```
In [19]: sample_variance = df['salary'].var(ddof=1)
print(f"Sample Variance: {sample_variance}")
sample_std = df['salary'].std(ddof=1)
print(f"Sample_std: {sample_std}")
```

Sample Variance: 474140633.88450044

Sample_std: 21774.770581673194

Exercise 4: Compute **Mean Absolute Deviation (MAD)** around the mean for `salary` .

```
In [20]: mean_salary = df['salary'].mean()
mad_salary = abs(df['salary'] - mean_salary).mean()
MAD= np.mean(abs(df['salary'] - mean_salary))
print(f"Mean Salary: {mean_salary}")
print(f"Mean Absolute Deviation: {mad_salary}")
print(f"MAD: {MAD}")
```

Mean Salary: 65316.229166666664

Mean Absolute Deviation: 15521.684722222222

MAD: 15521.684722222222

Exercise 5: Compute **Coefficient of Variation (CV)** for `salary` and `monthly_sales` (population std / mean).

```
In [21]: salary_CV = (df['salary'].std(ddof=0)/df['salary'].mean())
print(f"salary's CV is: {salary_CV}")
monthly_sales_CV = (df['monthly_sales'].std(ddof=0)/df['monthly_sales'].mean())
print(f"monthly_sales's CV is: {monthly_sales_CV}")
```

salary's CV is: 0.3326793264177604

monthly_sales's CV is: 0.8618341589620321

Exercise 6: By **department**, compute sample std dev and CV of `salary` .

```
In [28]: print("--- Computed the standard deviation for each department ---")
df.groupby('department')['salary'].std(ddof=1).reset_index()
```

--- Computed the standard deviation for each department ---

```
Out[28]:
```

	department	salary
0	Finance	24144.670400
1	HR	12643.570723
2	Operations	16920.694344
3	Sales	9531.186126
4	Tech	21230.000832


```
In [29]: print("--- Computed the CV for each department ---")
std = df.groupby('department')['salary'].std(ddof=1)
mean = df.groupby('department')['salary'].mean()
CV_for_departments = std/mean
CV_for_departments.reset_index()
```

--- Computed the CV for each department ---

```
Out[29]:
```

	department	salary
0	Finance	0.319773
1	HR	0.217513
2	Operations	0.273274
3	Sales	0.189135
4	Tech	0.253864

Using Dispersion to Compare Subsets

Exercise 1: Compare **Tech vs Sales**: compute mean, median, std dev of salary for each.

```
In [32]: df.head(3)
filtered_df = df[df['department'].isin(['Sales', 'Tech'])]
filtered_df.groupby('department')['salary'].agg(
    ['mean', 'median', 'std']
).reset_index()
```

```
Out[32]:
```

	department	mean	median	std
0	Sales	50393.481481	49984.0	9531.186126
1	Tech	83627.484375	80468.0	21230.000832

Exercise 2: Across **regions**, compute **CV of monthly_sales** and list regions by CV (desc).

```
In [33]: std_for_cv = df.groupby('region')['monthly_sales'].std()
mean_for_cv = df.groupby('region')['monthly_sales'].mean()
cv = std_for_cv/mean_for_cv
cv.sort_values(ascending=False)
```

```
Out[33]: region
South    1.011391
East     0.837225
West     0.777793
North    0.679524
Name: monthly_sales, dtype: float64
```

Exercise 3: For **managers vs non-managers**, compute mean & std of **salary**.

```
In [25]: df.groupby('is_manager')['salary'].agg(['mean', 'std']).reset_index()
```

```
Out[25]:
```

	is_manager	mean	std
0	0	63762.934884	22050.592692
1	1	78674.560000	13364.789583